Experiment No : 8

Title: Implement Binary search tree with operations Create, search, and recursive traversal.

Name of Student: Shamiksha Chaudhari
Class: SY **Div:** SY1 **PRN No:** B25ET1005 **Batch:** A1

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
   int data;
   struct node *left, *right;
};

struct node* create(int value)
{
   struct node *newnode;
   newnode = (struct node*)malloc(sizeof(struct node));

   newnode->data = value;
   newnode->left = NULL;
   newnode->right = NULL;

   return newnode;
}

struct node* insert(struct node *root, int value)
{
   if(root == NULL)
```

```
        return create(value);

    if(value < root->data)
        root->left = insert(root->left, value);
    else if(value > root->data)
        root->right = insert(root->right, value);

    return root;
}

void search(struct node *root, int key)
{
    if(root == NULL)
        printf("Element not found");
    else if(root->data == key)
        printf("Element found");
    else if(key < root->data)
        search(root->left, key);
    else
        search(root->right, key);
}

void inorder(struct node *root)
{
    if(root != NULL)
    {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
```

```
void preorder(struct node *root)
{
   if(root != NULL)
   {
      printf("%d ", root->data);
      preorder(root->left);
      preorder(root->right);
   }
}

void postorder(struct node *root)
{
   if(root != NULL)
   {
      postorder(root->left);
      postorder(root->right);
      printf("%d ", root->data);
   }
}

int main()
{
   struct node *root = NULL;
   int choice, value, key, n, i;

   do
   {
      printf("\n\n1. Create");
      printf("\n2. Search");
      printf("\n3. Inorder Traversal");
      printf("\n4. Preorder Traversal");
      printf("\n5. Postorder Traversal");
```

```
printf("\n6. Exit");

printf("\nEnter choice: ");
scanf("%d", &choice);

switch(choice)
{
   case 1:
      printf("Enter number of nodes: ");
      scanf("%d", &n);

      for(i = 0; i < n; i++)
      {
         printf("Enter value: ");
         scanf("%d", &value);
         root = insert(root, value);
      }
      break;

   case 2:
      printf("Enter element to search: ");
      scanf("%d", &key);
      search(root, key);
      break;

   case 3:
      printf("Inorder: ");
      inorder(root);
      break;

   case 4:
      printf("Preorder: ");
```

```
            preorder(root);
            break;

        case 5:
            printf("Postorder: ");
            postorder(root);
            break;

        case 6:
            printf("Exiting...");
            break;

        default:
            printf("Invalid choice");
      }
   } while(choice != 6);

   return 0;
}
```

Output :-

```
1. Create
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter choice: 1
Enter number of nodes: 3
Enter value: 45
Enter value: 65
Enter value: 76


1. Create
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter choice: 2
Enter element to search: 65
Element found

1. Create
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter choice: 3
Inorder: 45 65 76
```

```
1. Create
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter choice: 4
Preorder: 45 65 76

1. Create
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter choice: 5
Postorder: 76 65 45

1. Create
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter choice: 6
Exiting...
```